

INSTITUTO SUPERIOR TÉCNICO

MASTERS IN MECHANICAL ENGINEERING

OPTIMAL CONTROL LABORATORY PROJECT - PHASE III

PROFESSORS: JORGE MARTINS, RUI COELHO

Modeling and Simulation of a Quadcopter

103665 - Pedro Vasques

106598 - Joao Martinho

116570 - Melchior Poirier-Coutansais

119283 - Orestis Kourounakis

Contents

Contents	1
Introduction	2
1 Discrete-Time Controller and Observer	2
1.1 Discrete-Time LQR Servo Controller	2
1.1.1 Discrete-Time Plant (for Controller Design Only)	2
1.1.2 Augmented Servo Formulation	2
1.1.3 Gain Design via Discrete LQR	2
1.2 Discrete-Time Kalman Filter	3
1.2.1 Predict-Update Recursion	3
1.2.2 Comparison with the Continuous Kalman-Bucy Observer	3
1.3 Validation of Discrete-Time Controller and Observer - Nonlinear Model	4
1.3.1 Step Reference Validation	4
1.3.2 Complex Trajectory Validation	5
2 Control the Drone	7
2.1 Implementation of Discrete-Time Controller on the Parrot Mambo Fly - Simulation	7
2.2 Basic Flight Tasks: Laboratory Experiment	18
2.2.1 Initial Controller Implementation & Tuning	18
2.2.2 Practical Observations	23
2.2.3 Reference Tracking	23
2.3 Complex Trajectory Tracking	26
Conclusion	28

Introduction

1 Discrete-Time Controller and Observer

1.1 Discrete-Time LQR Servo Controller

1.1.1 Discrete-Time Plant (for Controller Design Only)

Each continuous-time subsystem is discretised via Zero-Order Hold (ZOH) with sampling period $T_s = 5$ ms to obtain the plant matrices needed for controller design. Considering a ZOH at the input, $u(t) = u(kT_s)$ for $kT_s \leq t < (k+1)T_s$, the solution of the continuous-time state equation between $t = kT_s$ and $t = (k+1)T_s$ yields the discrete-time equivalents:

$$\bar{A} = e^{A_c T_s}, \quad \bar{B} = \int_0^{T_s} e^{A_c \lambda} B_c d\lambda, \quad \bar{C} = C_c, \quad \bar{D} = D_c, \quad (1)$$

giving the discrete-time state-space model:

$$x[k+1] = \bar{A}x[k] + \bar{B}u[k], \quad y[k] = \bar{C}x[k] + \bar{D}u[k]. \quad (2)$$

This yields four discrete pairs $(\bar{A}_i, \bar{B}_i)$ for the altitude, lateral-roll, longitudinal-pitch, and yaw subsystems. These are used only to compute the regulator gains; the continuous-time plant in Simulink is unchanged. For this first part, the Kalman-Bucy observer from Phase II is also retained without modification.

1.1.2 Augmented Servo Formulation

To ensure zero steady-state error to step references, each subsystem is augmented with an integrator state $x_{i,i}$ driven by the tracking error, following the Forward Euler discretisation of a continuous integrator:

$$x_{i,i}[k+1] = x_{i,i}[k] + T_s e_i[k] = x_{i,i}[k] + T_s (r_i - C_{\text{out},i} \hat{x}_i[k]), \quad (3)$$

where r_i is the step reference command for subsystem i , $C_{\text{out},i}$ is the first row of C_i selecting the position output, and $\hat{x}_i[k]$ is the state estimate provided by the Kalman Filter. The augmented open-loop system is then:

$$\begin{bmatrix} x_i[k+1] \\ x_{i,i}[k+1] \end{bmatrix} = \underbrace{\begin{bmatrix} \bar{A}_i & 0 \\ -T_s C_{\text{out},i} & 1 \end{bmatrix}}_{\bar{A}_{a,i}} \begin{bmatrix} x_i[k] \\ x_{i,i}[k] \end{bmatrix} + \underbrace{\begin{bmatrix} \bar{B}_i \\ 0 \end{bmatrix}}_{\bar{B}_{a,i}} u_i[k] + \begin{bmatrix} 0 \\ T_s \end{bmatrix} r_i, \quad (4)$$

and the control law applied at each sample instant is:

$$u_i[k] = - \begin{bmatrix} K_{d,i} & K_{\text{id},i} \end{bmatrix} \begin{bmatrix} \hat{x}_i[k] \\ x_{i,i}[k] \end{bmatrix}. \quad (5)$$

The integral action guarantees zero steady-state tracking error for step references, as an asymptotically stable regulator on the augmented system drives all states to their steady-state values.

1.1.3 Gain Design via Discrete LQR

The gains $[K_{d,i}, K_{\text{id},i}]$ are found by solving the Discrete Algebraic Riccati Equation (DARE) on the augmented system. This is implemented via value iteration (`my_dare.m`), equivalent to MATLAB's built-in `d1qr()` but without toolbox dependencies. The infinite-horizon quadratic cost function minimised is:

$$J = \sum_{k=0}^{\infty} [\bar{x}_{a,i}^\top[k] Q_i \bar{x}_{a,i}[k] + u_i^\top[k] R_i u_i[k]], \quad (6)$$

where $\bar{x}_{a,i} = [x_i^\top \ x_{i,i}^\top]^\top$ is the augmented state. The optimal gain is the solution of the DARE:

$$P = \bar{A}_{a,i}^\top P \bar{A}_{a,i} - \bar{A}_{a,i}^\top P \bar{B}_{a,i} (R_i + \bar{B}_{a,i}^\top P \bar{B}_{a,i})^{-1} \bar{B}_{a,i}^\top P \bar{A}_{a,i} + Q_i, \quad (7)$$

from which:

$$K_{a,i} = (R_i + \bar{B}_{a,i}^\top P \bar{B}_{a,i})^{-1} \bar{B}_{a,i}^\top P \bar{A}_{a,i}. \quad (8)$$

The weighting matrices are set following Bryson's rule, an established heuristic from optimal control where each diagonal entry of Q_i is set to the reciprocal square of the maximum acceptable deviation for that state:

$$Q_1 = \text{diag} \left(\frac{1}{r_{3,\max}^2}, \frac{1}{v_{3,\max}^2}, \frac{1}{\xi_{1,\max}^2} \right), \quad (9)$$

with $r_{3,\max} = 2 \text{ m}$, $v_{3,\max} = 2 \text{ m/s}$, $\xi_{1,\max} = 1 \text{ m.s}$. Analogous matrices are defined for the remaining subsystems. The scalar control weight is $R_i = \rho$ uniformly across all subsystems, chosen such that the closed-loop poles lie strictly inside the unit disc. The Kalman-Bucy observer from Phase II is replaced by a discrete-time Kalman Filter in the following subsection.

1.2 Discrete-Time Kalman Filter

1.2.1 Predict-Update Recursion

The continuous Kalman-Bucy observer from Phase II is replaced by a discrete-time Kalman filter, implemented using the Simulink `Kalman Filter` block. Consider the discrete-time stochastic system:

$$x[k+1] = \bar{A}x[k] + \bar{B}u[k] + w[k], \quad y[k] = Cx[k] + v[k], \quad (10)$$

where $w[k] \sim \mathcal{N}(0, Q_0)$ is the process noise and $v[k] \sim \mathcal{N}(0, R_0)$ is the measurement noise, both assumed zero-mean Gaussian white noise processes, uncorrelated with each other. The Kalman filter minimising prediction-error covariance runs as a two-step recursion at each sample instant k :

$$\text{Predict: } \hat{x}_{k+1} = \bar{A} \hat{x}_k + \bar{B} u_k, \quad (11)$$

$$P_{k+1} = \bar{A} P_k \bar{A}^\top + Q_0, \quad (12)$$

$$\text{Update: } L_k = P_{k-1} C^\top (C P_{k-1} C^\top + R_0)^{-1}, \quad (13)$$

$$\hat{x}_k = \hat{x}_{k-1} + L_k [y_k - C \hat{x}_{k-1}], \quad (14)$$

$$P_k = (I - L_k C) P_{k-1}. \quad (15)$$

The same noise covariance matrices Q_0 and R_0 from Phase II are reused. Since $T_s = 5 \text{ ms}$ is small, the continuous covariances are used directly as an approximation of the discrete equivalents. The process noise input matrix is set to $G_d = I$, per the MATLAB convention that process noise enters all states directly. At steady state, L_k converges to a constant L ; the filter then becomes a time-invariant discrete observer. The stationary Kalman filter gain L is obtained by solving the discrete algebraic Riccati equation:

$$P_\infty = \bar{A} P_\infty \bar{A}^\top - \bar{A} P_\infty C^\top (C P_\infty C^\top + R_0)^{-1} C P_\infty \bar{A}^\top + Q_0, \quad (16)$$

from which:

$$L = P_\infty C^\top (C P_\infty C^\top + R_0)^{-1}. \quad (17)$$

The Simulink block is configured with the discrete plant matrices $(\bar{A}_i, \bar{B}_i, C_i)$ and the covariances Q_0, R_0 for each subsystem. The block option *Use current measurement $y[n]$ to improve $\hat{x}[n]$* was enabled, so the filter performs a full predict-update at each step.

1.2.2 Comparison with the Continuous Kalman-Bucy Observer

- **Time propagation.** The Kalman-Bucy observer integrates a continuous ODE at every solver step. The discrete filter only executes the predict-update recursion once per T_s , which is more efficient and directly suited to sampled-data systems.
- **Noise parameters.** The Kalman-Bucy filter uses continuous power spectral densities Q_0 and R_0 . The discrete filter requires their discrete-time equivalents; the process noise covariance becomes $G_d Q_0 G_d^\top$, which depends on T_s .

- **Riccati equation.** The Kalman-Bucy gain solves the continuous ARE; the discrete filter solves the DARE.
- **Initial conditions.** The discrete filter requires an explicit initial state estimate $\hat{x}[0]$ and initial covariance $P[0]$, whereas the continuous observer only requires $\hat{x}(0)$.
- **Consistency with the controller.** Since the regulator already operates at $T_s = 5$ ms, using a discrete filter at the same rate avoids the need to interface a continuous observer with a discrete controller. The overall design is then fully discrete on both the estimation and control sides.

1.3 Validation of Discrete-Time Controller and Observer - Nonlinear Model

The complete discrete-time system (discrete LQR servo regulator combined with the discrete Kalman filter) was validated on the nonlinear Simulink model. The simulation used a fixed-step `ode4` solver at $h = 0.5$ ms to correctly resolve the continuous nonlinear plant driven by discrete control inputs, with both measurement and process noise active.

1.3.1 Step Reference Validation

The reference commands were simple step inputs applied to each subsystem individually, and the resulting state responses are shown in Figures 1–4.

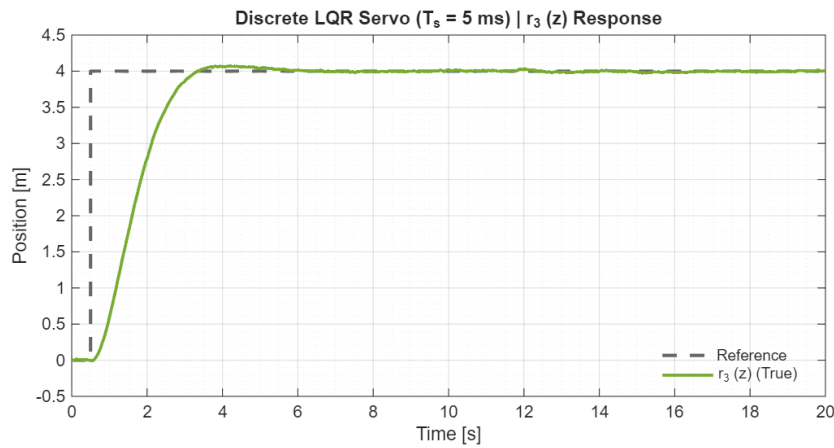


Figure 1: Vertical position r_3 step response — discrete LQR servo with discrete Kalman Filter on nonlinear model.

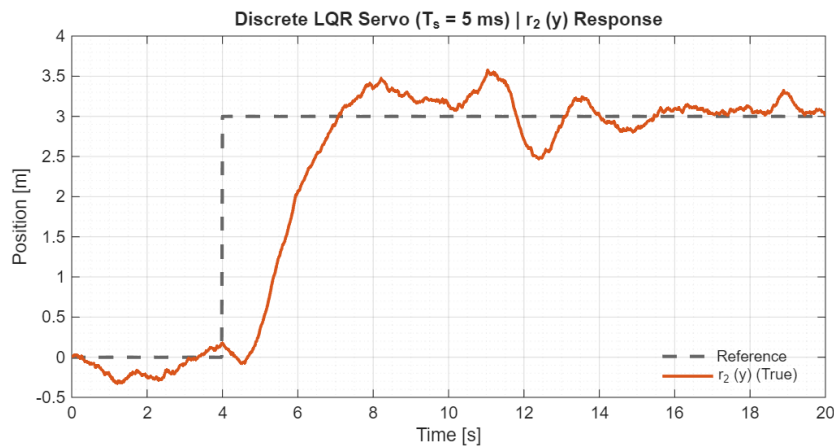


Figure 2: Lateral position r_2 step response — discrete LQR servo with discrete Kalman Filter on nonlinear model.

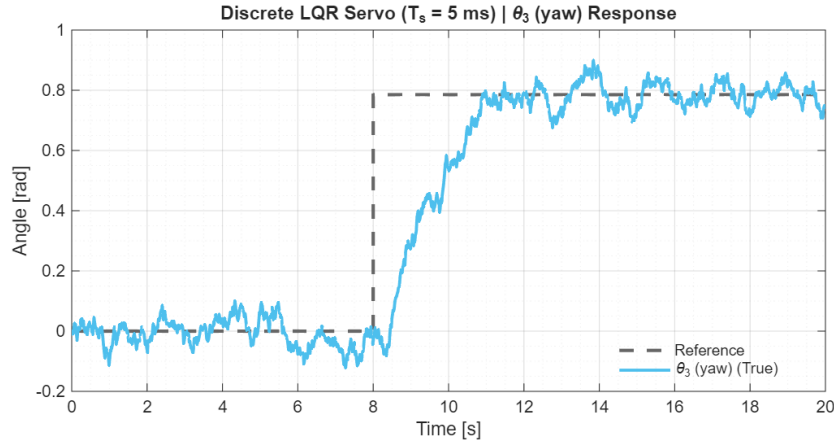


Figure 3: Yaw angle θ_3 step response — discrete LQR servo with discrete Kalman Filter on nonlinear model.

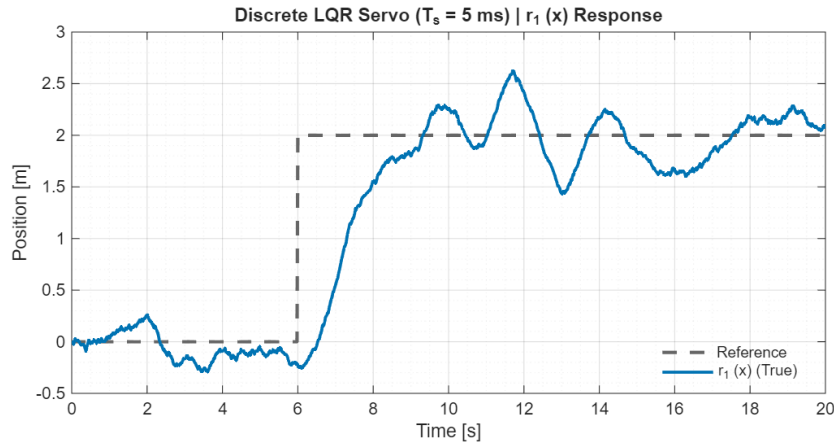


Figure 4: Longitudinal position r_1 step response — discrete LQR servo with discrete Kalman Filter on nonlinear model.

All four subsystems converge to their respective references with no steady-state error confirming the integral action. The vertical response is the smoothest, while the lateral and longitudinal channels show slight overshoot due to attitude coupling. The yaw channel presents more noise at steady state, as expected from its higher measurement noise variance. The discrete Kalman filter removes the noise across all channels, with estimates that closely track the true states.

1.3.2 Complex Trajectory Validation

A figure-eight (Lissajous) trajectory was used to test the discrete system under more demanding conditions. Defining $\tau = t - 5$ s as the time offset after the initial climb phase, the reference trajectory is:

$$r_1^{\text{ref}}(t) = A \sin(\omega\tau), \quad r_2^{\text{ref}}(t) = B \sin(2\omega\tau), \quad r_3^{\text{ref}}(t) = \min(5, 1.25t) \text{ m}, \quad (18)$$

with $A = 2$ m, $B = 1.5$ m, and $\omega = 2\pi \times 0.1$ rad/s. The drone first climbs to 5 m before the lateral pattern begins. The resulting 3D trajectory and state time histories are shown in Figures 5–8.

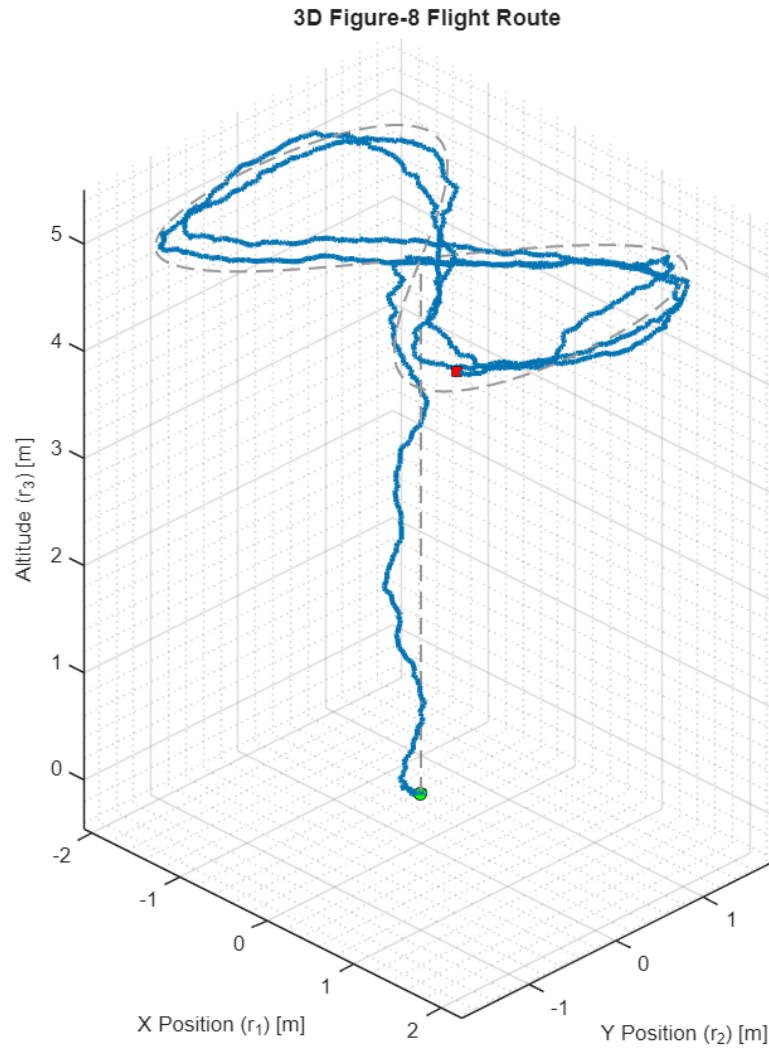


Figure 5: 3D trajectory of figure-eight Lissajous route — discrete LQR servo with discrete Kalman Filter on nonlinear model.

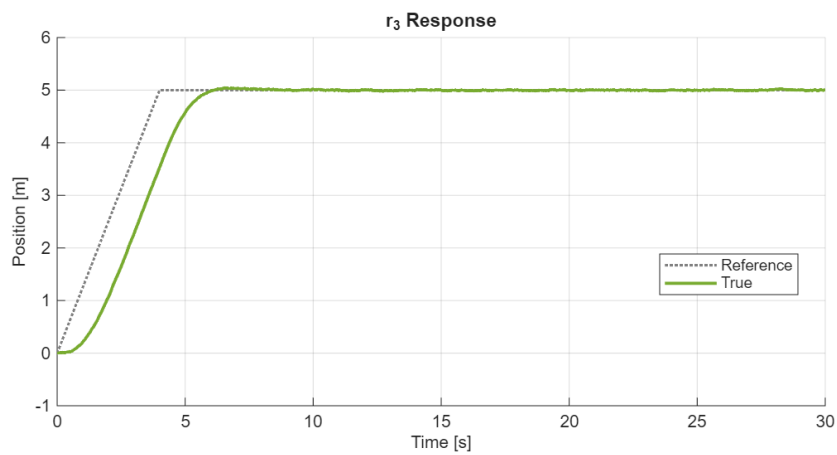


Figure 6: Vertical position r_3 tracking the Lissajous climb reference.

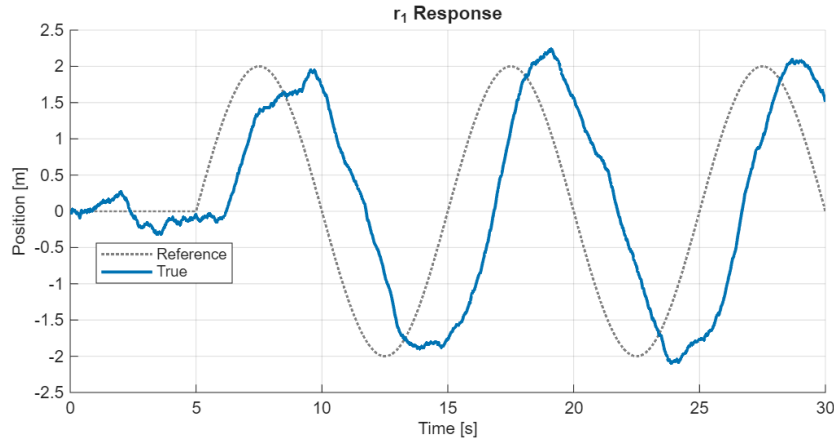


Figure 7: Longitudinal position r_1 tracking the Lissajous reference.

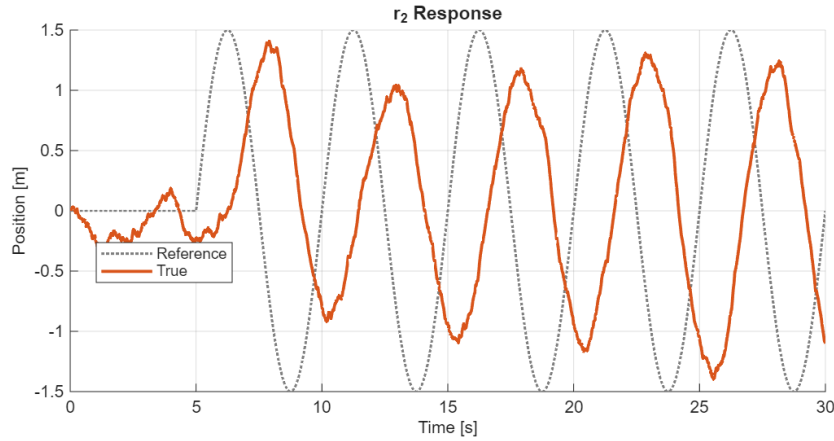


Figure 8: Lateral position r_2 tracking the Lissajous reference.

The drone tracks the figure-eight trajectory with minor phase lag on the faster channels. The results show that the discrete controller and observer handle continuous nonlinear dynamics and noise without degradation.

2 Control the Drone

2.1 Implementation of Discrete-Time Controller on the Parrot Mambo Fly - Simulation

To validate the discrete-time LQR servo controller and Kalman Filter under real flight conditions, the controller was integrated into the official `flightControlSystem.slx` Simulink environment provided by the faculty. This framework interfaces directly with the Parrot Mambo Fly hardware, replacing the built-in PID controllers with the discrete-time LQR servo controllers designed in Section 1.1.

To isolate axis-coupling effects and avoid closed-loop divergence during initial testing, a systematic decoupled calibration strategy was adopted. Controllers were enabled one subsystem at a time, while the remaining subsystems retained the baseline PID controllers. This approach allowed any performance deficiencies (tracking lag, steady-state error, or actuator oscillations) to be traced directly to the weighting matrices of the corresponding subsystem.

The deployment followed a strict subsystem replacement hierarchy, dictated by the plant's cascading aerodynamic dependencies:

1. **Vertical Subsystem (F_3):** Calibrated first to establish stable hover at a fixed altitude. A well-tuned vertical controller is the prerequisite for all subsequent tests.
2. **Yaw Subsystem (M_3):** Calibrated second to lock the drone's heading in the horizontal plane. Stabilising the yaw prevents heading drift from coupling into the position channels.
3. **Longitudinal Subsystem (M_2):** Calibrated third to control forward and backward displacement (r_1).

4. **Lateral Subsystem (M_1):** Calibrated last to govern side-to-side displacement (r_2).

For each subsystem, two flight tests were performed. The first test consisted of takeoff, hover, and land, to assess hover stability. The second test added a reference step in the controlled direction, to assess tracking performance. The exception was the vertical subsystem, for which the takeoff test already constitutes a full validation of altitude tracking. The flight results for each subsystem are shown in Figures 9 through 15.

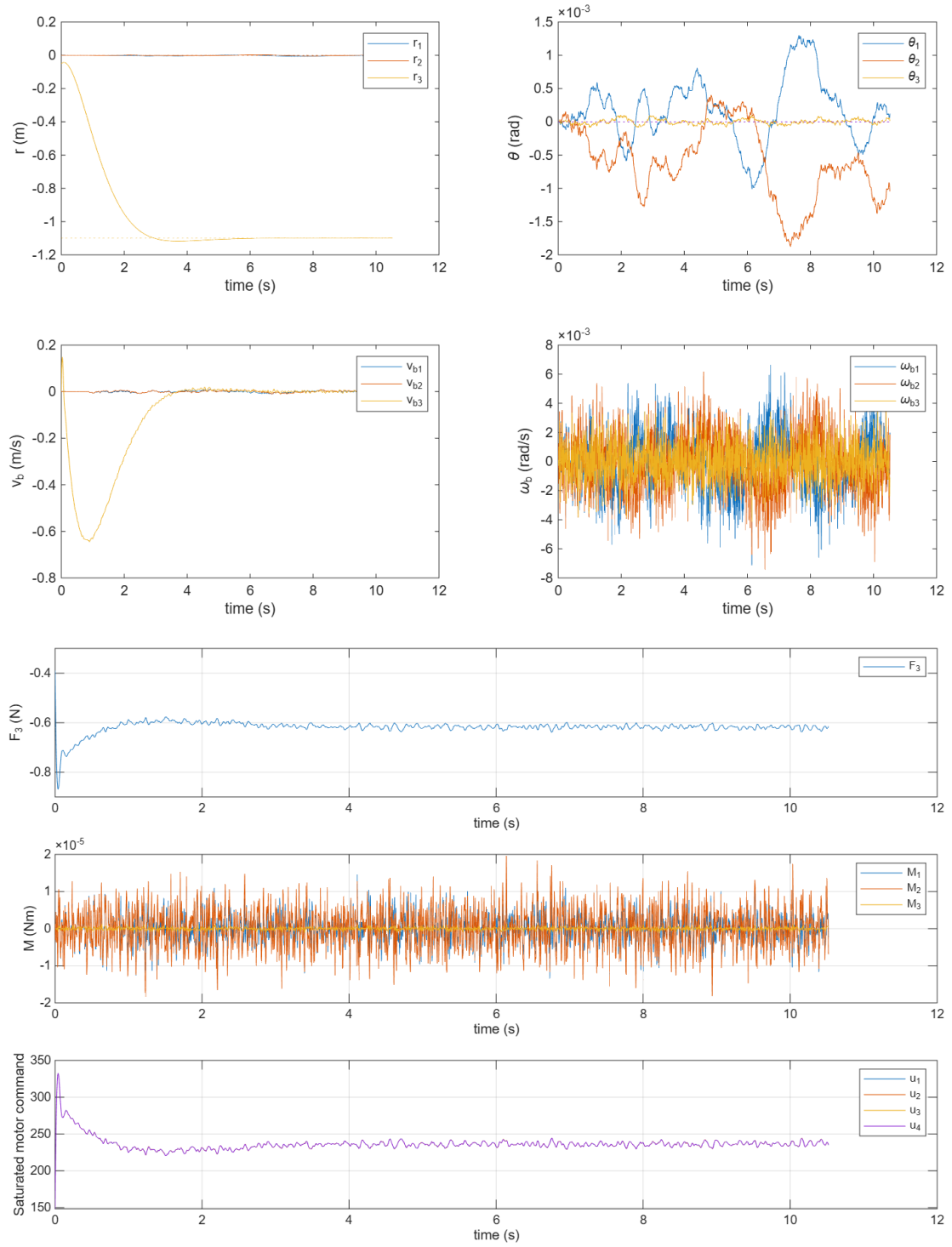


Figure 9: Vertical subsystem (F_3): takeoff and hover flight test.

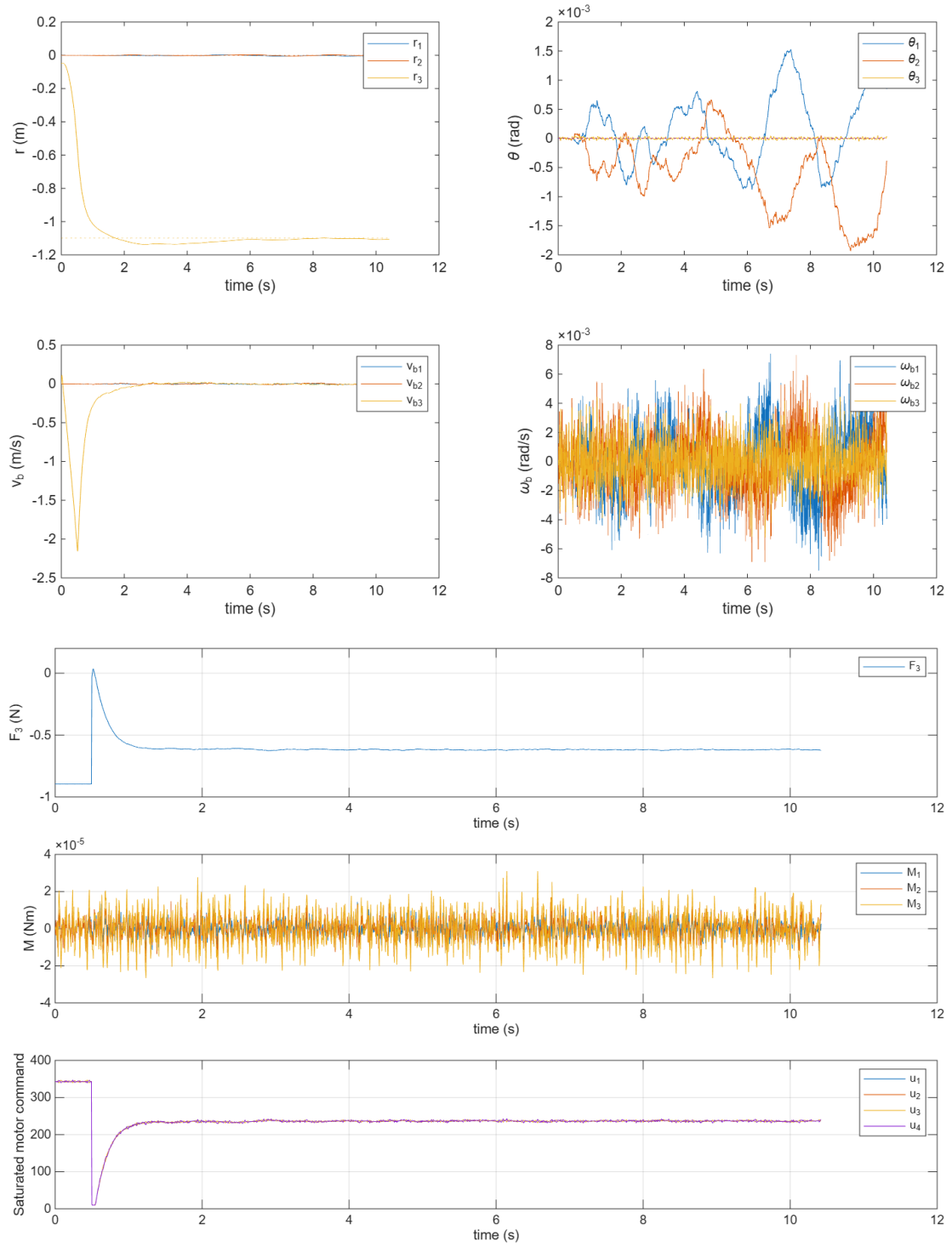


Figure 10: Yaw subsystem (M_3): takeoff and hover flight test.

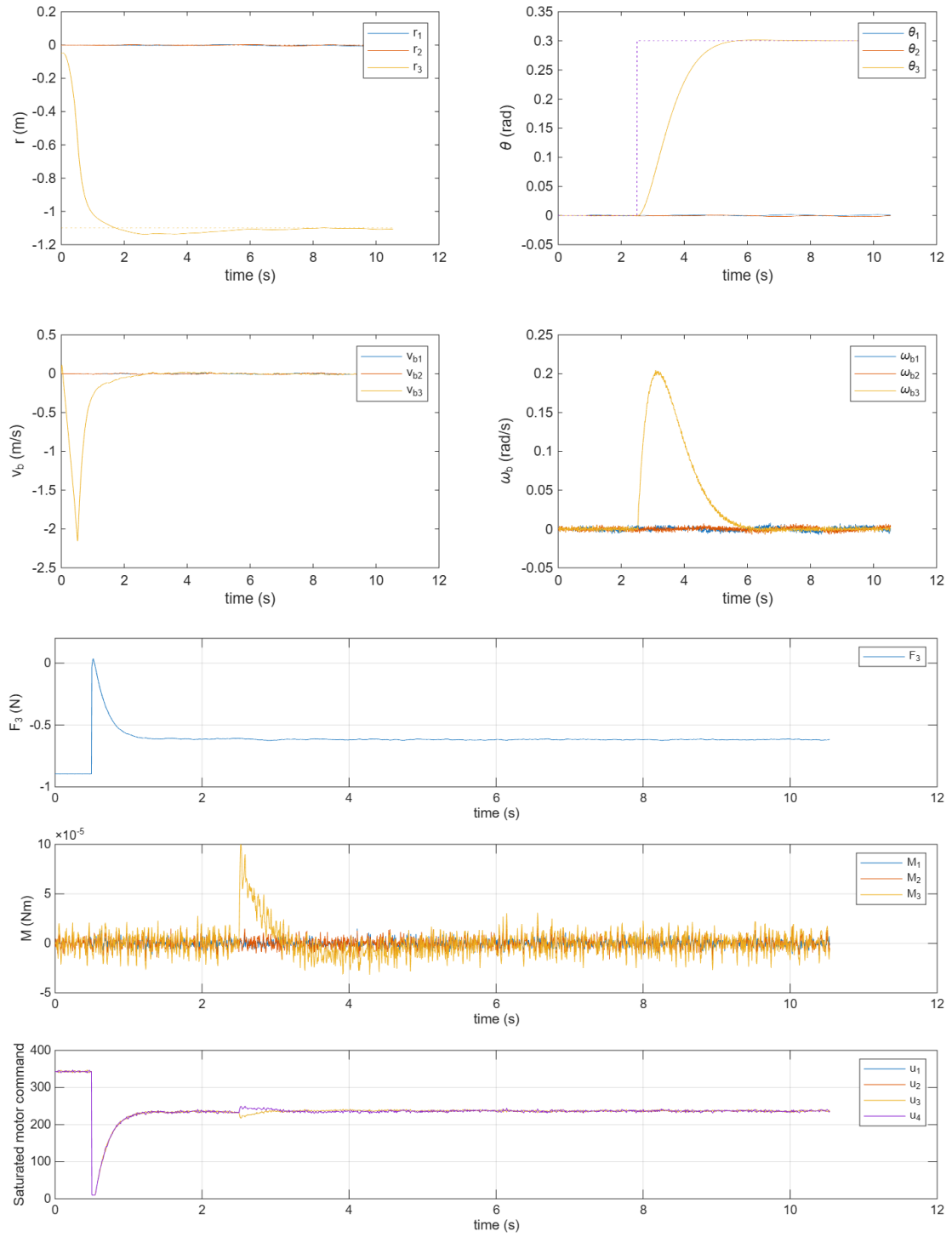


Figure 11: Yaw subsystem (M_3): takeoff and yaw reference tracking.

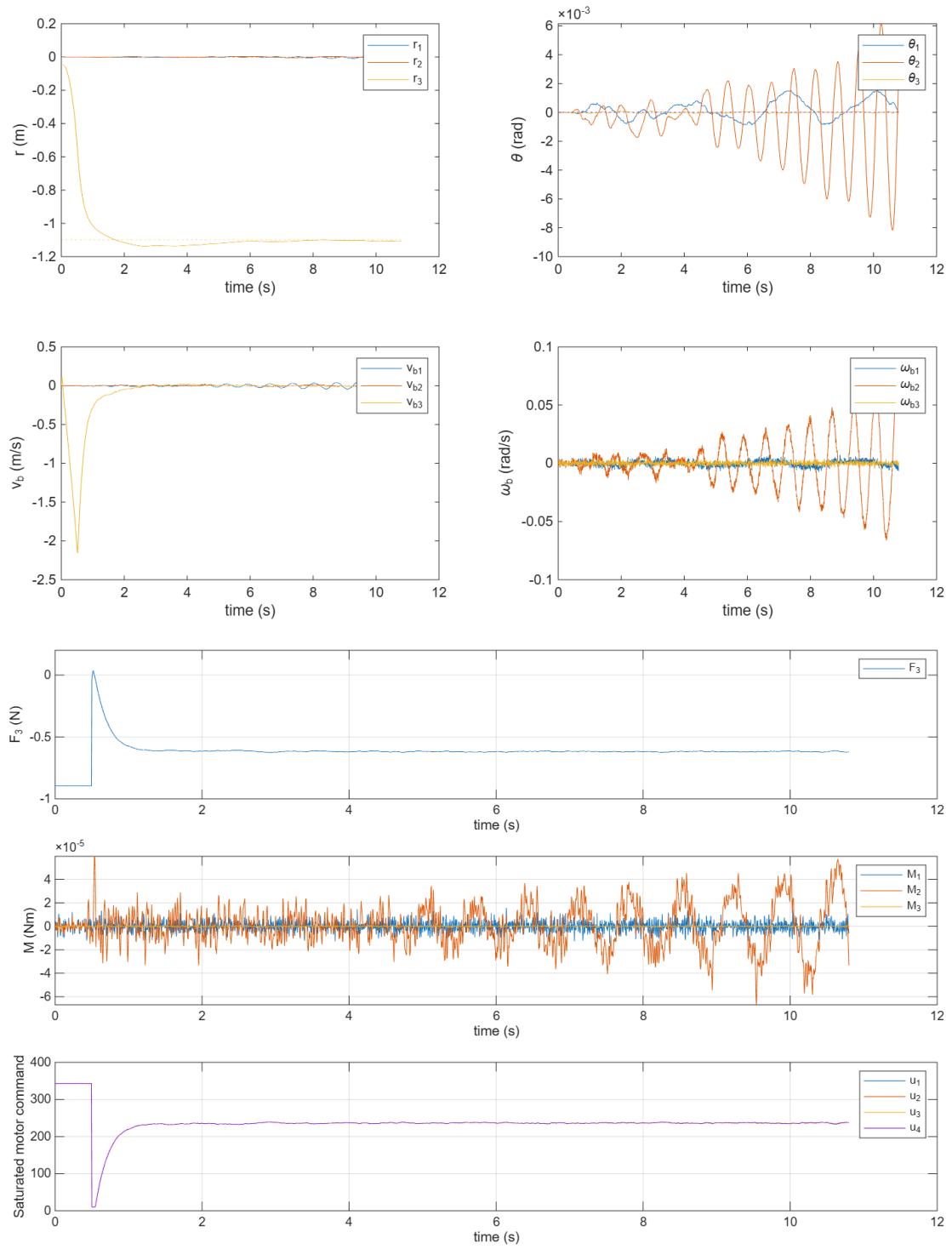


Figure 12: Longitudinal subsystem (M_2): takeoff and hover flight test.

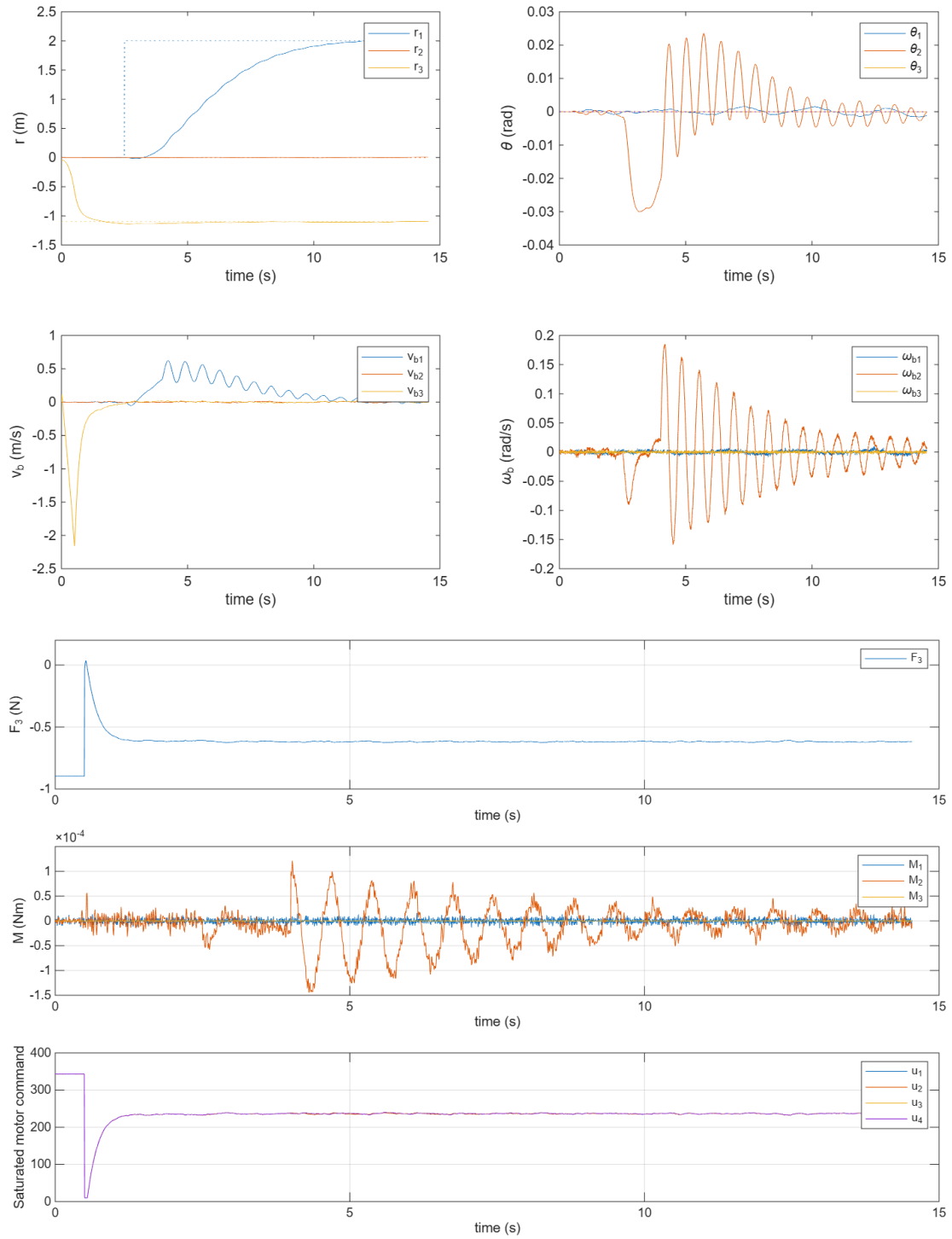


Figure 13: Longitudinal subsystem (M_2): takeoff and r_1 reference tracking.

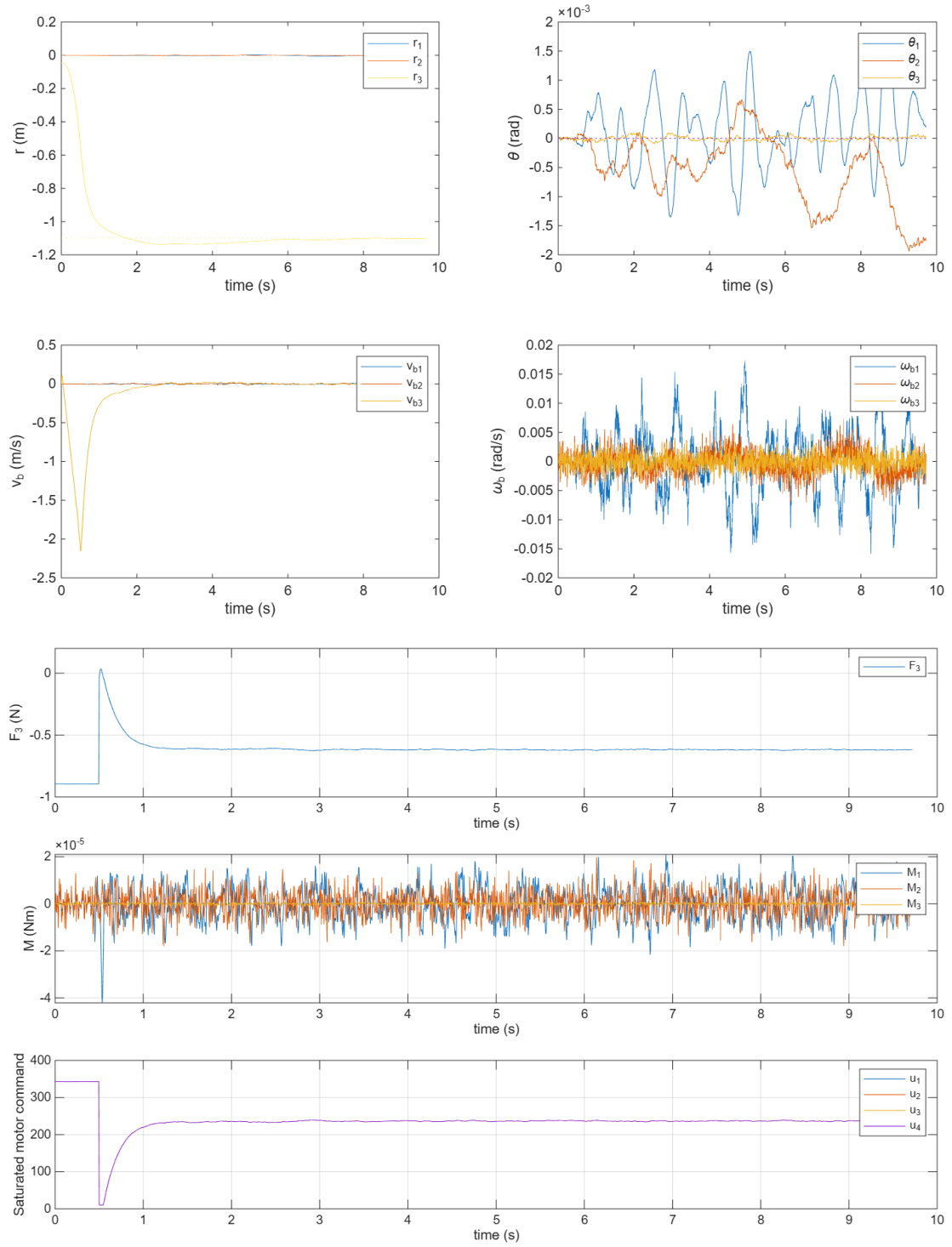


Figure 14: Lateral subsystem (M_1): takeoff and hover flight test.

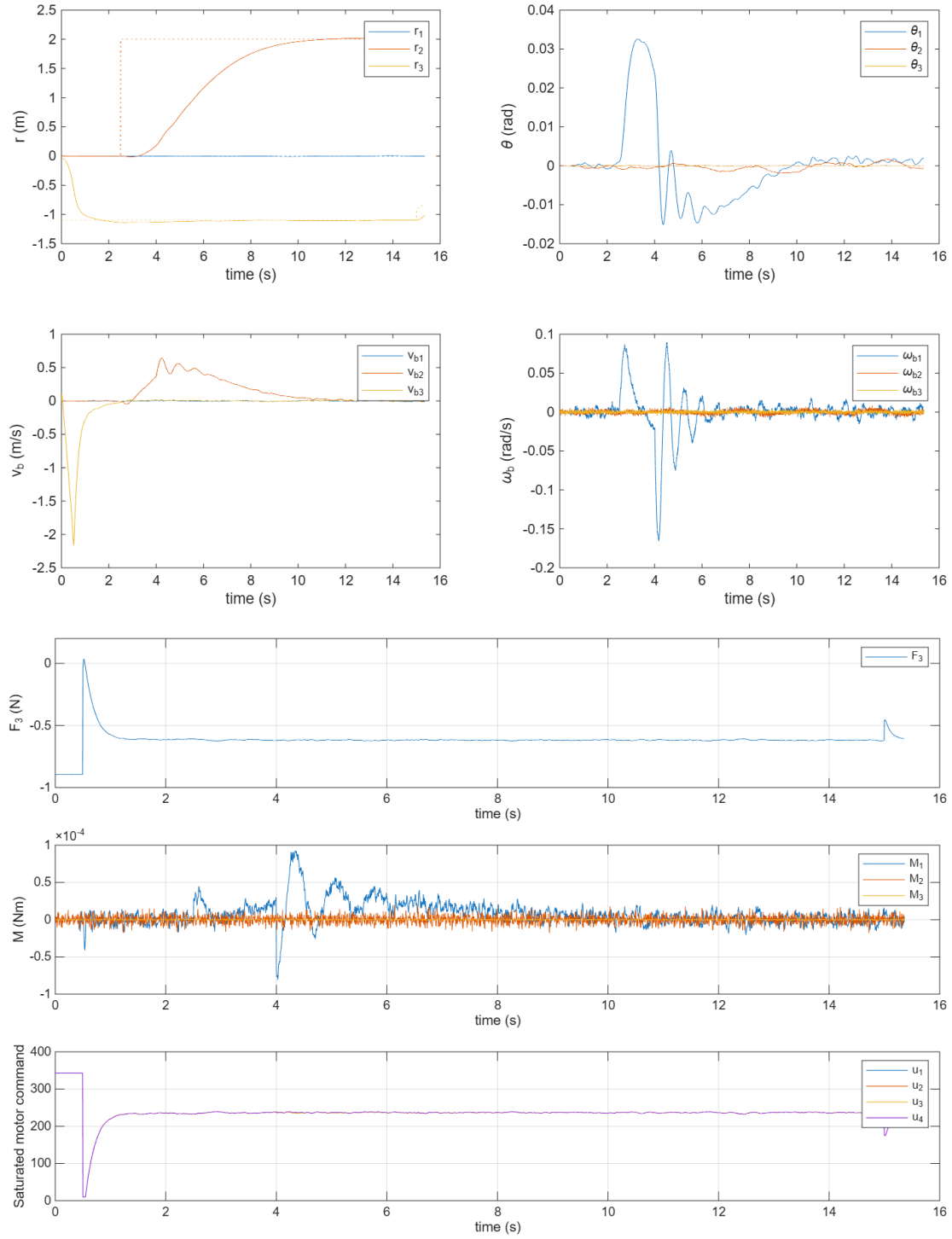


Figure 15: Lateral subsystem (M_1): takeoff and r_2 reference tracking.

Once all four subsystems were individually validated, all controllers were activated simultaneously. Two additional flight tests were performed: a takeoff-and-hover test to assess combined stability, and a full reference tracking test with simultaneous references on all subsystems. The results are shown in Figures 16 and 17.

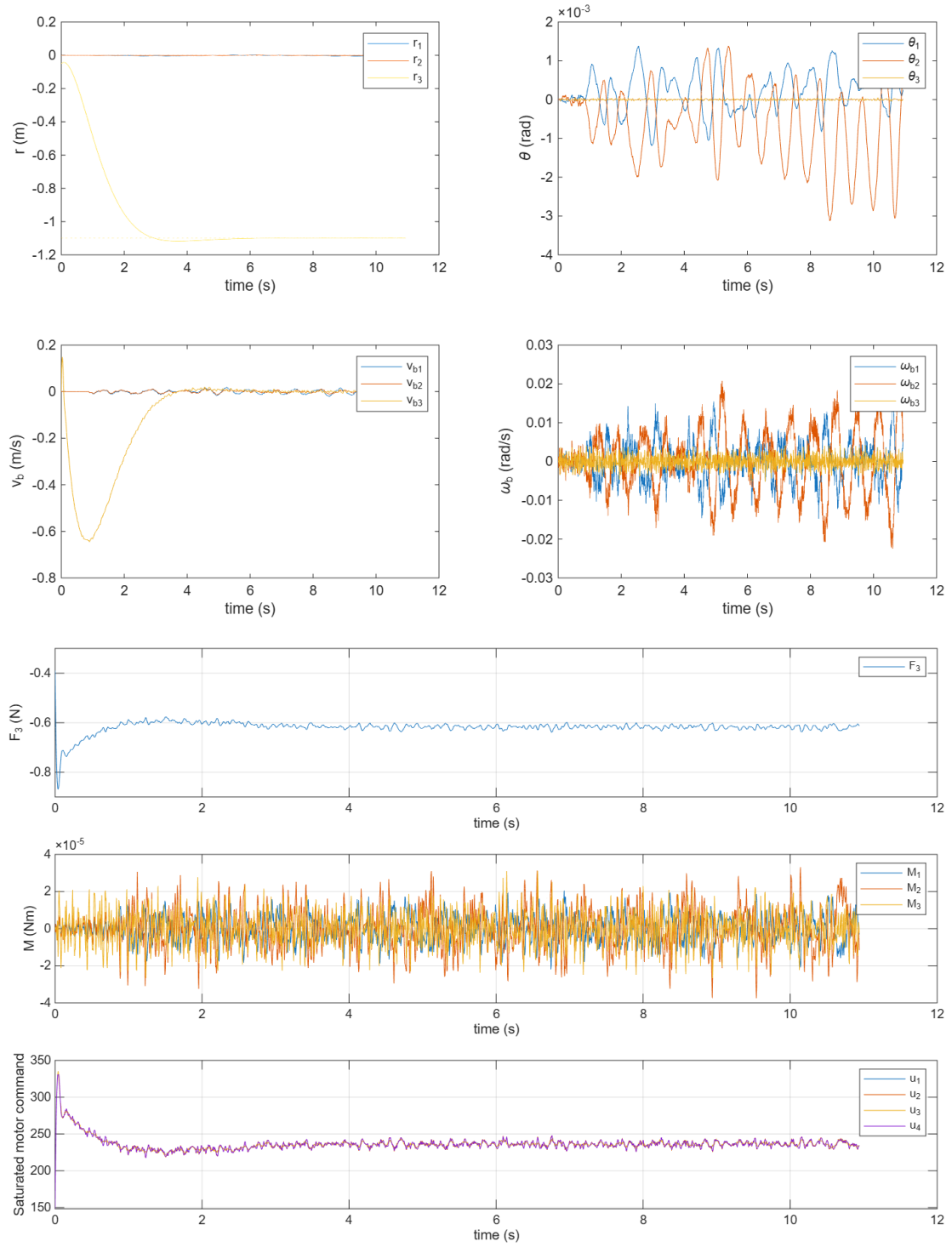


Figure 16: All subsystems active: takeoff and hover flight test.

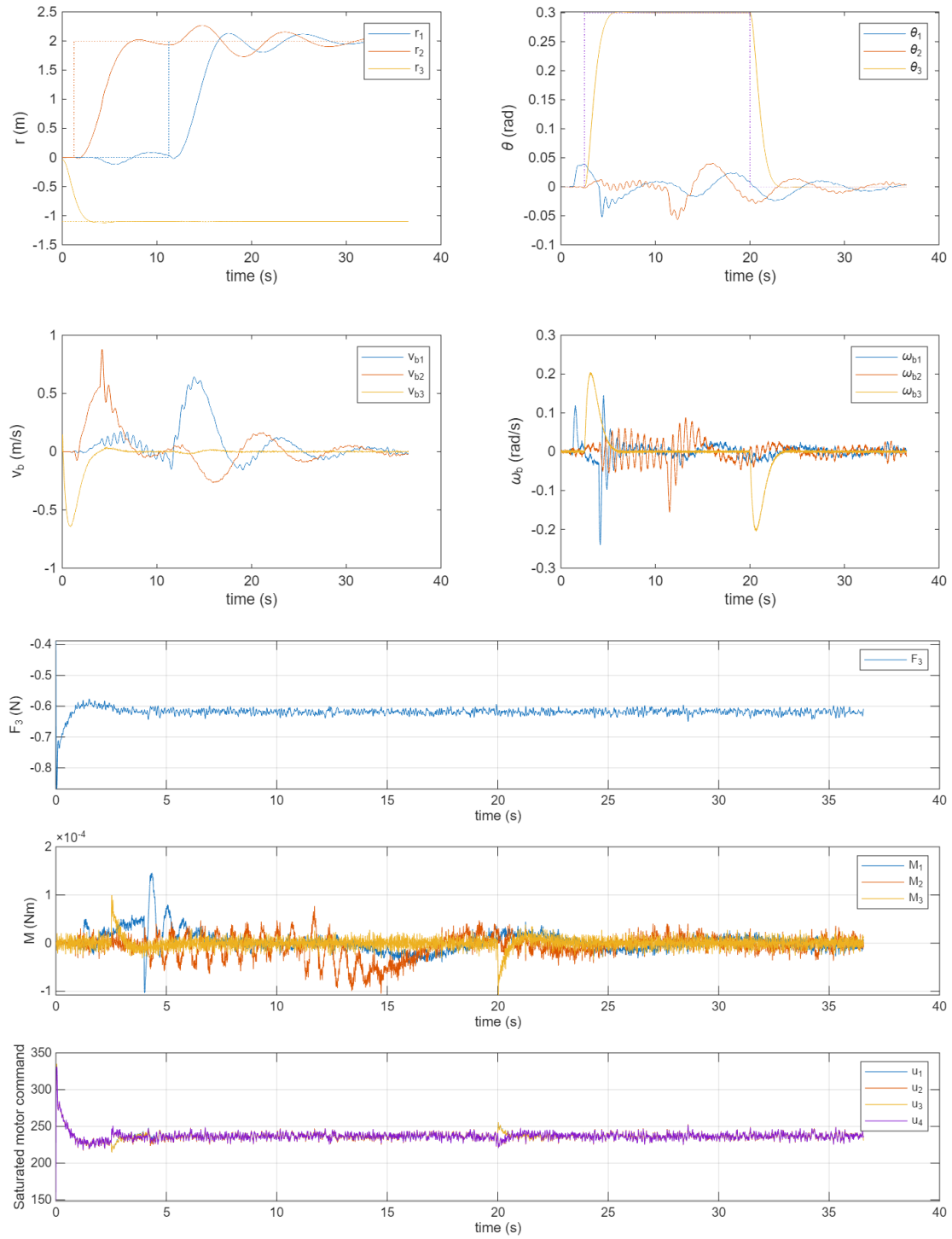


Figure 17: All subsystems active: simultaneous reference tracking on all axes.

The simulation results confirmed that all four subsystems track their respective references without steady-state error and with acceptable transient behaviour, validating the controller gains before proceeding to the physical drone experiments.

2.2 Basic Flight Tasks: Laboratory Experiment

2.2.1 Initial Controller Implementation & Tuning

Having validated the controller gains in the simulation environment, the same decoupled calibration strategy was applied to the physical Parrot Mambo Fly drone, following the same subsystem replacement hierarchy established in the simulation phase.

Before deploying the custom controllers, the drone was first flown with the default PID controller to identify any hardware biases. Small but consistent offsets were observed in the lateral and longitudinal positions at takeoff, arising from imperfections in the drone's physical symmetry and propeller calibration. Identifying these offsets before deploying the custom controllers allowed the reference values to be adjusted accordingly, so the drone did not drift away from the intended vertical trajectory at takeoff.

The controllers were then enabled one subsystem at a time, while the remaining subsystems retained the baseline PID controllers. Once a subsystem controller was validated at hover, it was kept active and the next subsystem controller was introduced. The primary tuning parameter at this stage was ρ . Increasing it penalises control effort more heavily; the response is gentler, which helps avoid oscillations but risks making the controller too sluggish to reject disturbances. Decreasing it makes the controller more aggressive, which improves tracking speed but can cause actuator saturation and oscillatory behaviour in the attitude channels. The progressive deployment followed the same four-stage hierarchy as the simulation phase, with the flight results shown in Figures 18 through 21.

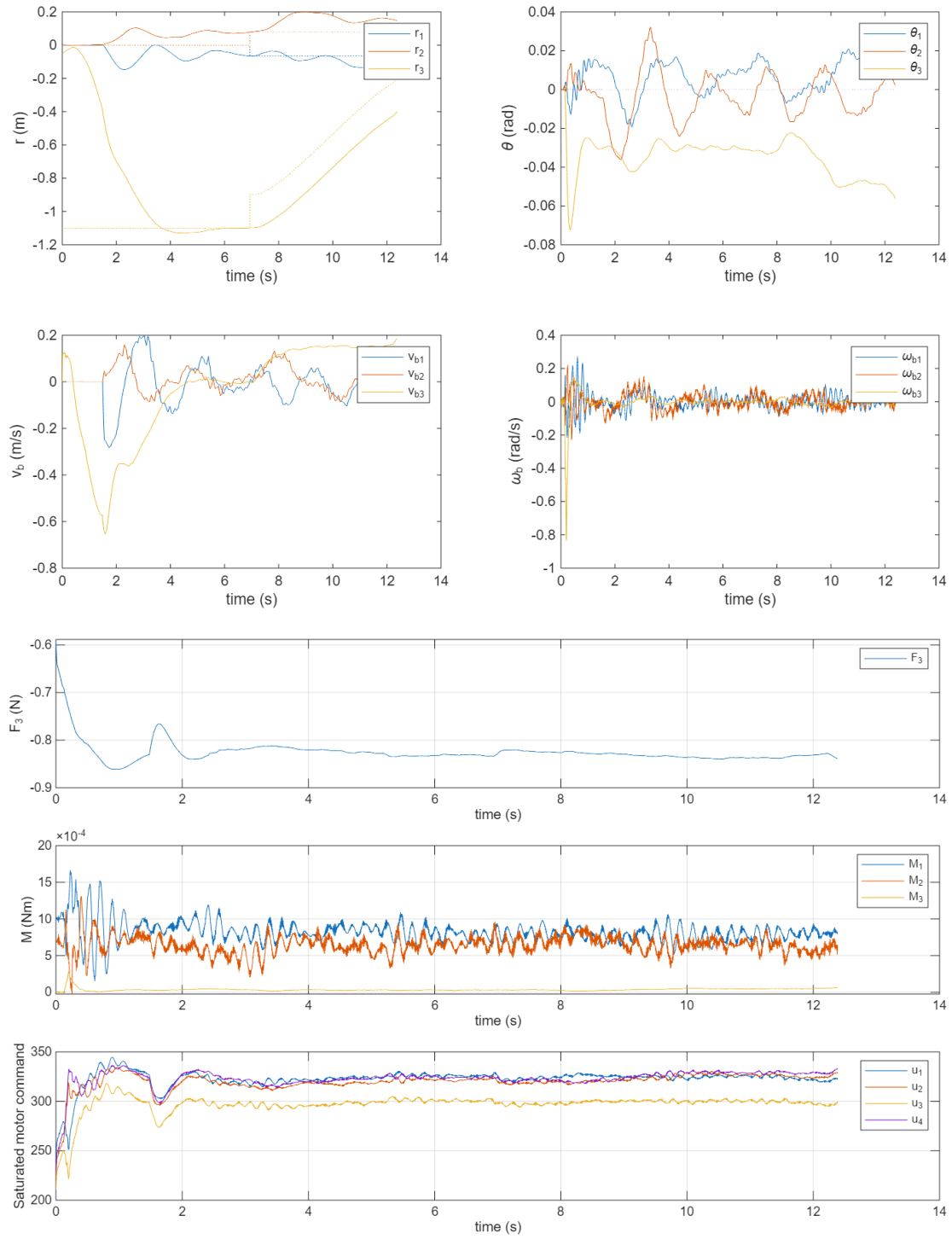


Figure 18: Vertical subsystem (F_3): takeoff and hover flight test.

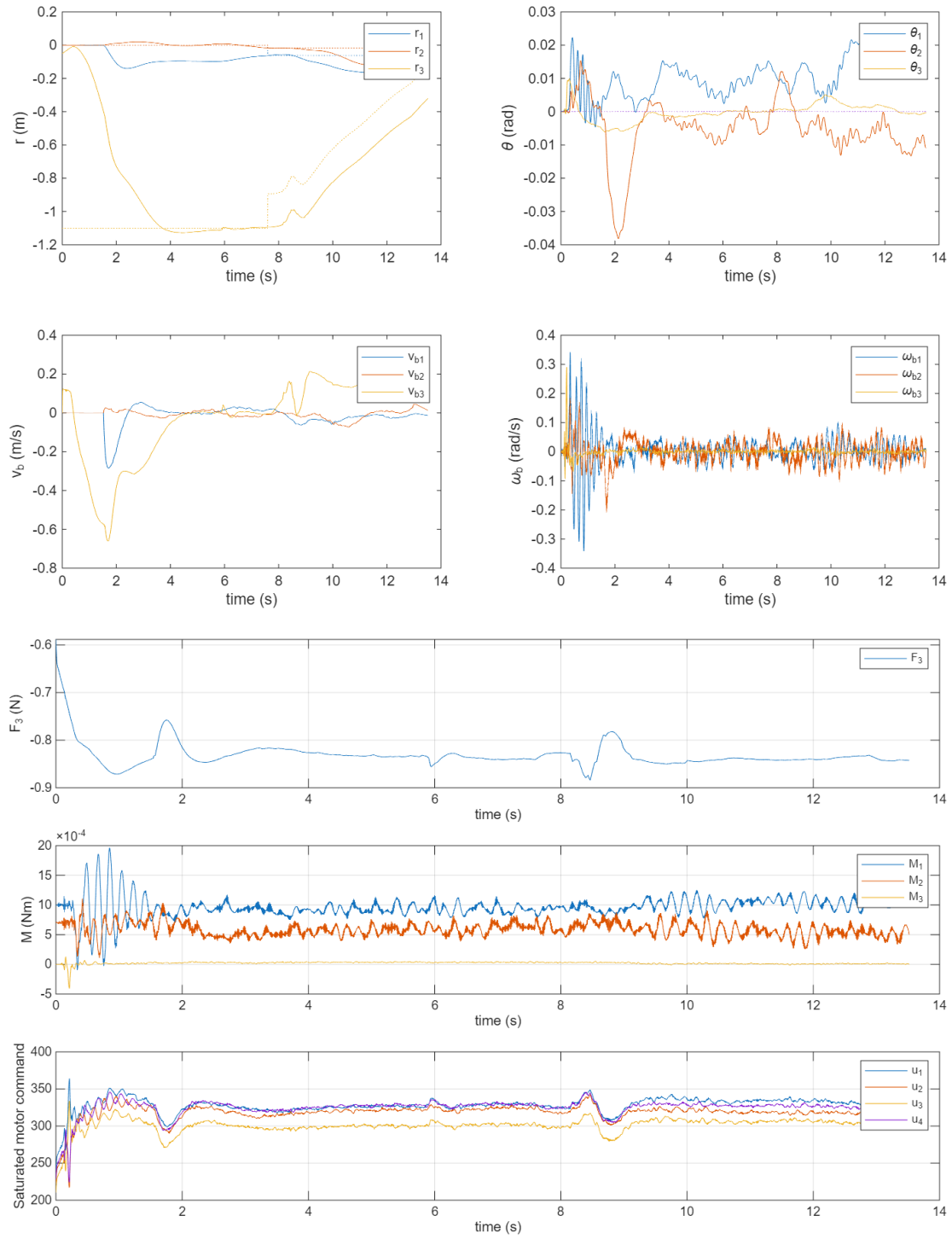


Figure 19: Yaw subsystem (M_3): takeoff and hover flight test.

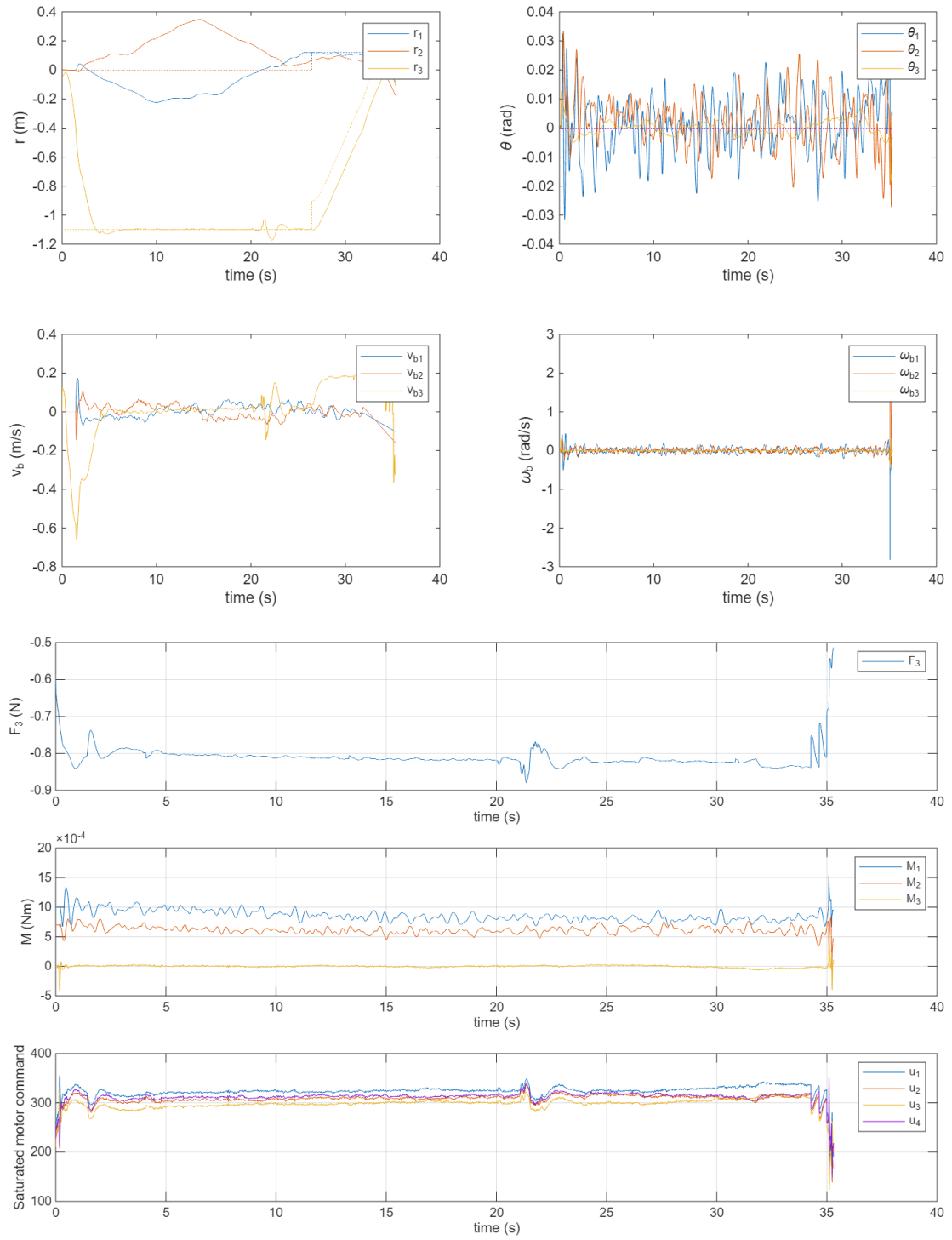


Figure 20: Longitudinal subsystem (M_2): takeoff and hover flight test.

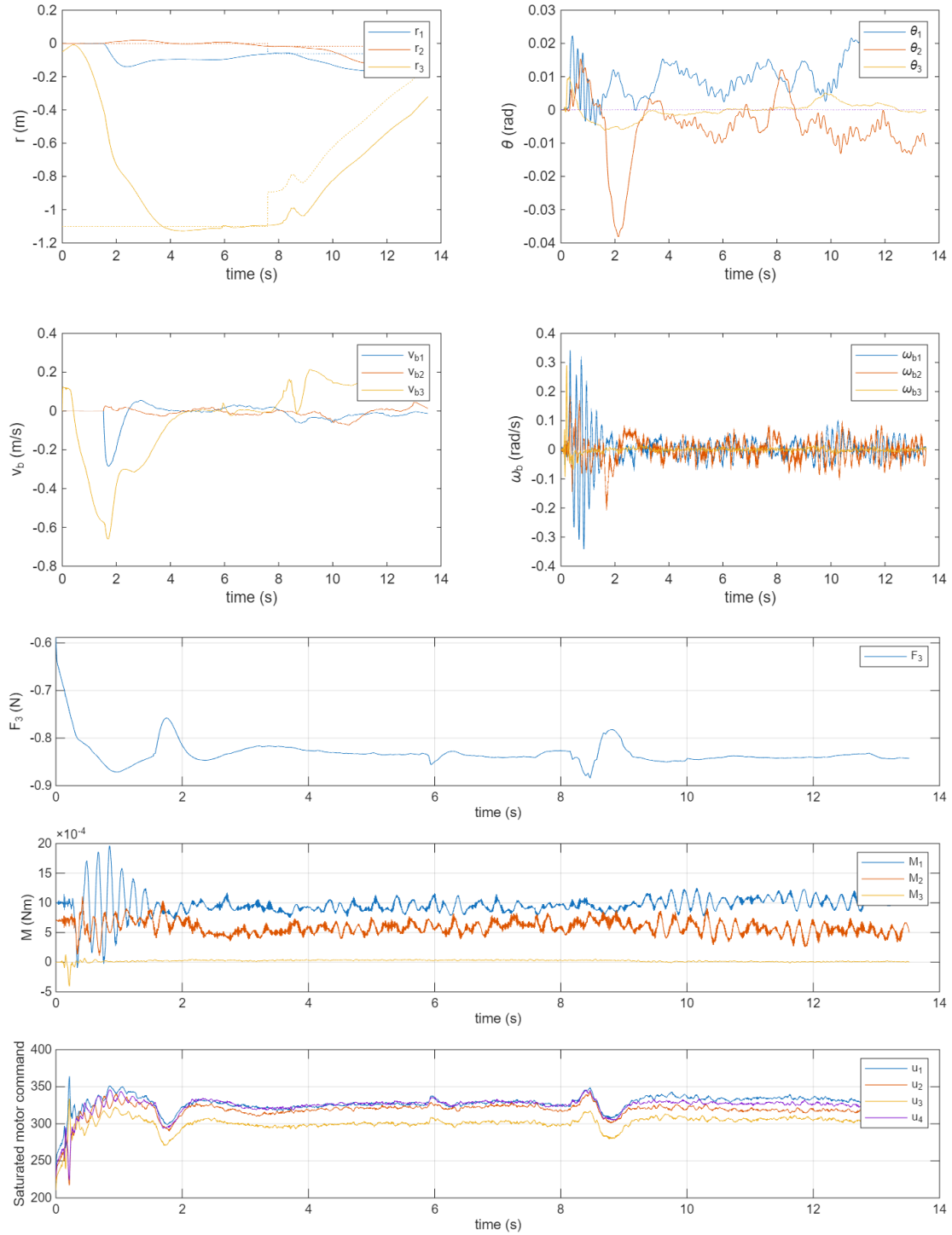


Figure 21: Lateral subsystem (M_1): takeoff and hover flight test.

The progressive deployment confirmed that each subsystem controller performed correctly in isolation before being combined with the others. The vertical subsystem achieved stable hover with minimal altitude deviation. The yaw subsystem locked the heading effectively and reduced lateral drift from uncontrolled rotation. The longitudinal and lateral subsystems tracked their respective position references with acceptable transient behaviour. With all four controllers active simultaneously, the drone maintained stable hover and tracked simultaneous references on all axes.

2.2.2 Practical Observations

Throughout the hardware experiments, several practical observations were made that were important for reliable flight:

- **Hardware checks before each flight.** The propellers should spin freely; they can seize the wings get locked or if debris are present. The rubber feet of the drone should all be correctly mounted and symmetrically positioned, as a misaligned foot shifts the takeoff reference and introduces an initial bias that the controller must compensate.
- **Lateral or longitudinal drift at takeoff.** If the drone shivered or drifted sideways immediately after takeoff rather than climbing vertically, the hardware biases were re-checked and the reference offsets updated accordingly.
- **Loss of vertical reference.** On some occasions the drone lost its altitude reference mid-flight, likely due to momentary loss of the downward-facing optical flow sensor signal caused by insufficient floor texture or lighting conditions. Ensuring adequate floor contrast and lighting reduced the frequency of this issue.

2.2.3 Reference Tracking

Once all four subsystems were validated at hover, reference ramp inputs were applied to each subsystem to assess tracking performance. Three movement tests were performed, each isolating one position channel while the remaining subsystems maintained hover:

1. **Vertical movement:** the drone climbed to a reference altitude r_3 and stabilised into hover.
2. **Longitudinal movement:** the drone translated forward to a reference r_1 and stabilised.
3. **Lateral movement:** the drone moved sideways to a reference r_2 and stabilised.

At this stage it became apparent that adjusting individual entries of the Q_i matrix (rather than the global scalar ρ) offered finer control over the drone's behaviour. Each diagonal entry of Q_i penalises the deviation of the corresponding state:

- **Position state weights** (e.g. r_3, r_1, r_2, θ_3). Increasing these makes the controller more aggressive in correcting position error, at the cost of larger attitude angles and motor commands.
- **Velocity state weights** (e.g. $\dot{r}_3, \dot{r}_1, \dot{r}_2, \dot{\theta}_3$). Increasing these penalises fast motion; the response is smoother but slower.
- **Integral state weights** (ξ_i). Increasing these speeds up the elimination of steady-state error but can cause overshoot if set too high.

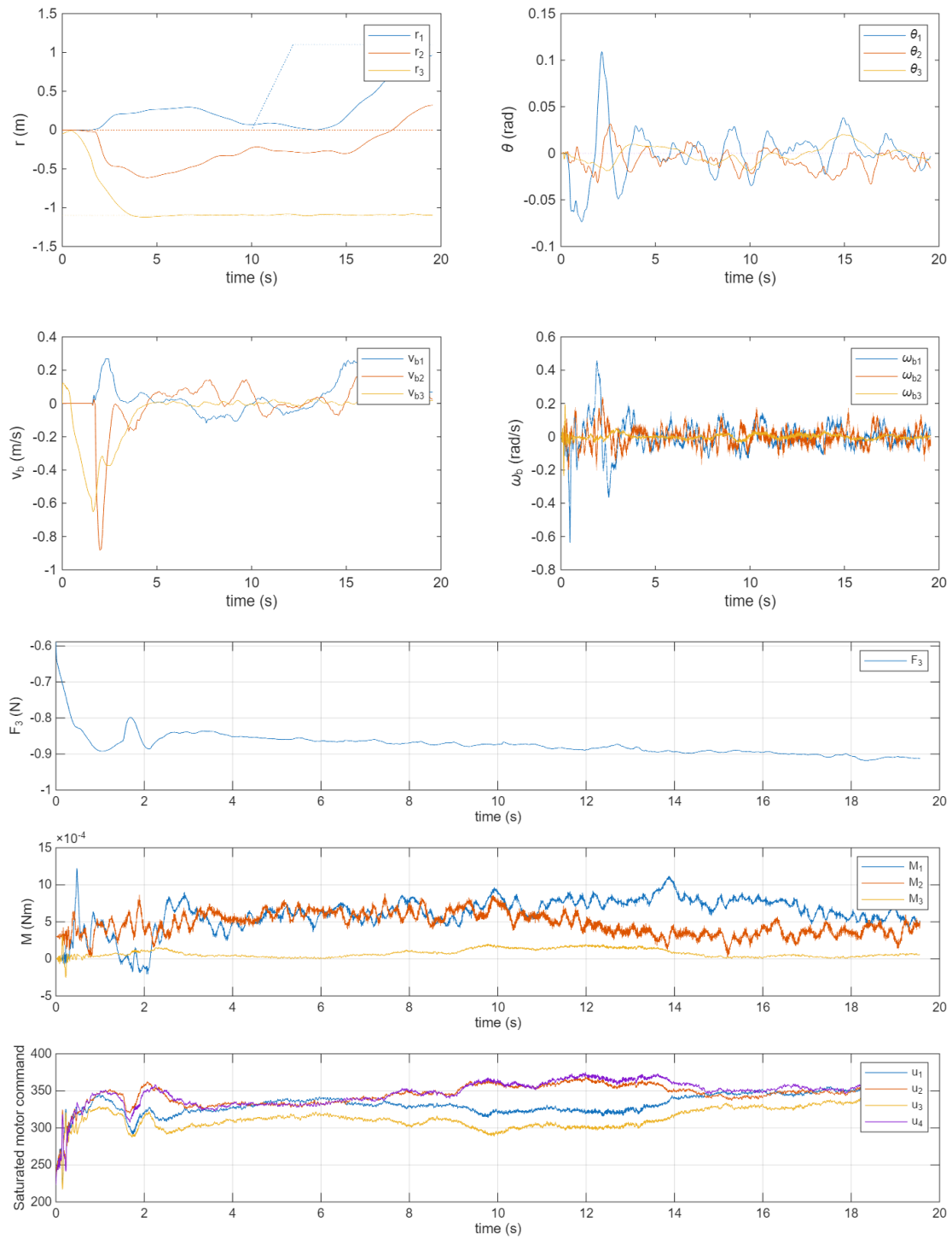


Figure 22: Longitudinal subsystem (M_2): takeoff and r_1 reference tracking.

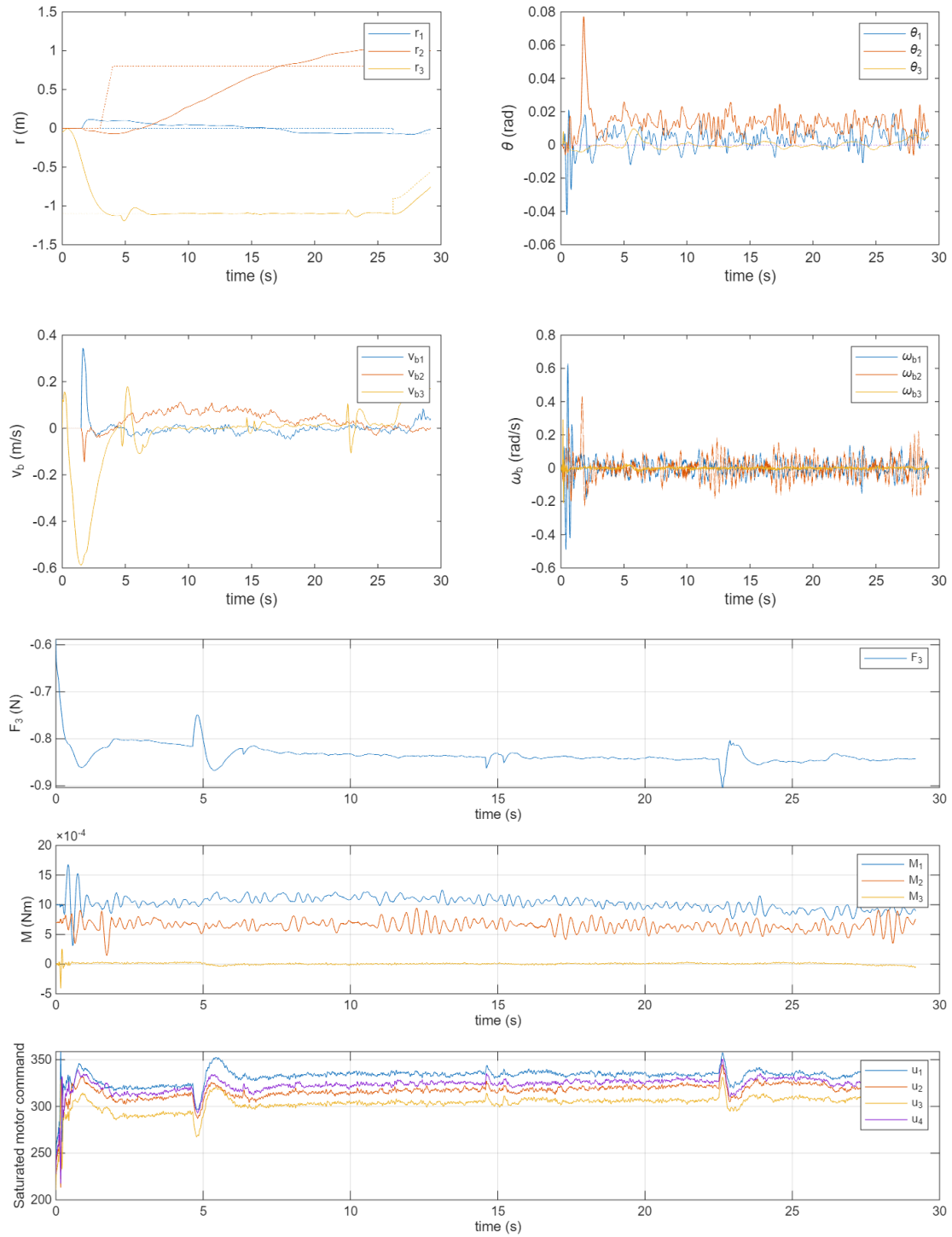


Figure 23: Lateral subsystem (M_1): takeoff and r_2 reference tracking.

As it is possible to see both controllers go to the reference but with a really considerable delay, and with a overshoot on the M_1 controller, so we keep trying to get a better result. And we achieve it but unfortunately we hadn't time to retake these plots, and it is only possible to see that results on the subsection 2.3

2.3 Complex Trajectory Tracking

To further show the robustness of our controller we did a multi axis movement at the same time.

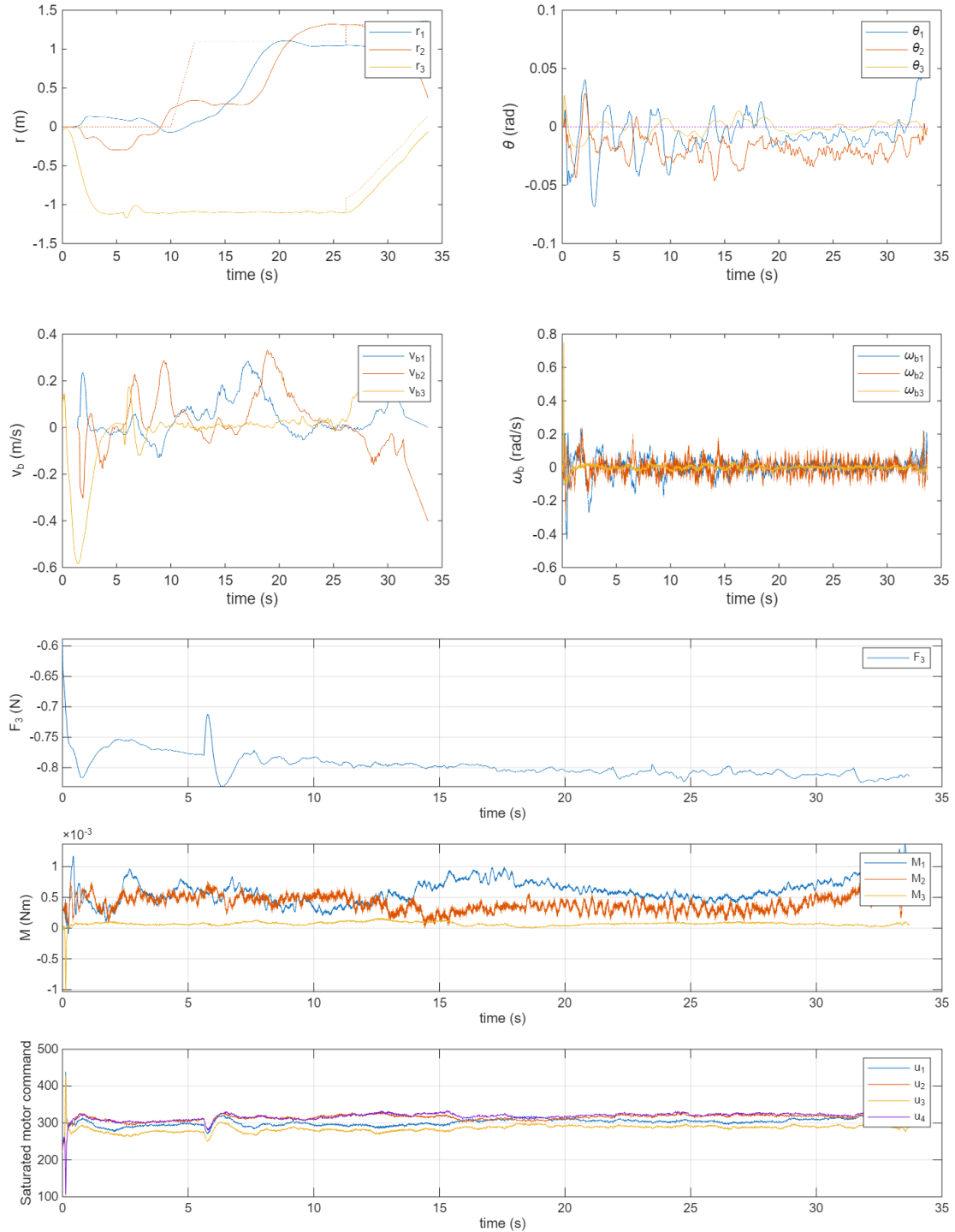


Figure 24: Harder route. Move on two axes on same time

We weren't happy with these results so we tried with a small change on the angular velocity on M_1 and M_2 .

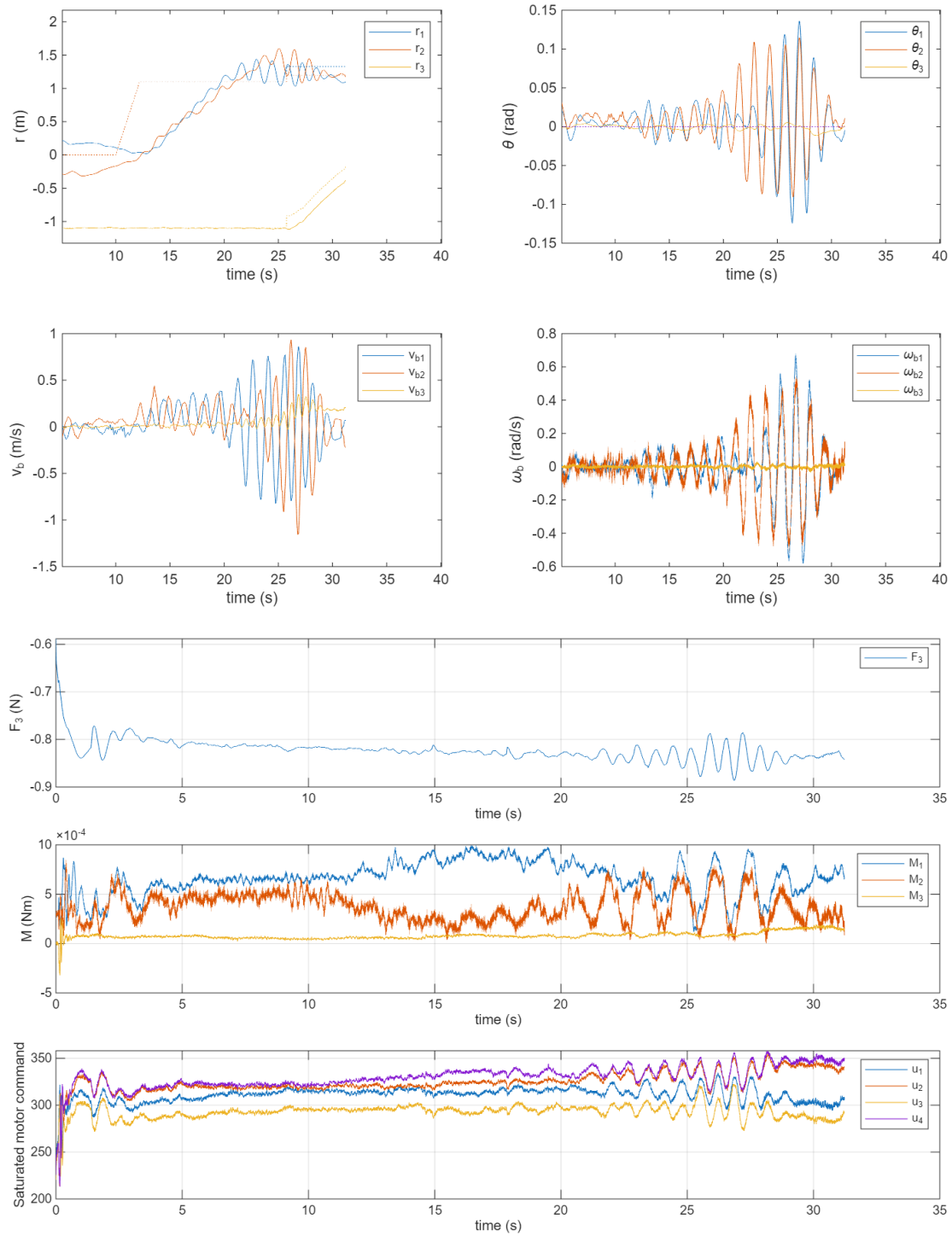


Figure 25: Harder route. Move on two axes on same time. Recalibration of M_1 and M_2

We achieved with these change a more similar movement on both axis but it got worse on hovering. So we decide to use the previous controller as our final controller.

Conclusion

In this phase, the continuous-time LQR servo controller and Kalman-Bucy observer developed in Phase II were extended to discrete time and validated both in simulation and on the physical Parrot Mambo Fly drone.

The discrete-time LQR servo controller, designed via Zero-Order Hold discretisation at $T_s = 5$ ms, and the discrete-time Kalman Filter were first validated on the nonlinear simulation model, confirming correct tracking behaviour in the presence of noise and nonlinear coupling.

On the physical drone, a decoupled calibration strategy was adopted, enabling one subsystem at a time. The scalar ρ proved most effective during the initial hover tuning phase, while fine-tuning individual Q_i entries offered more precise control during reference tracking. Practical hardware checks — propeller condition, foot alignment, and floor contrast for the optical flow sensor — proved essential to consistent flight performance.

To achieve a better tracking response, two primary adjustments should be done:

- **Relax Attitude Constraints:** Reduce proportional roll/pitch gains to allow the drone to tilt further, translating aerodynamic thrust into faster lateral acceleration.
- **Adjust Integral Action:** Increase the integral control term to reduce response latency and improve the system's ability to eliminate tracking errors during dynamic maneuvers.

The drone successfully completed all required basic flight tasks and demonstrated complex trajectory tracking, confirming the validity of the discrete-time LQG design on physical hardware.